

Build AI Agents That Survive Failure





Nikolay Advolodkin

Staff Developer Advocate @ Temporal

Dog Dad & Roller Skater



@nikolay_a00



/in/nikolayadvolodkin/



AGENDA

- ✓ Foundations of Agentic Systems
- ✓ Building with an Agent Framework, but Durable
- ✓ Humans in the Loop (HITL)
- ✓ Orchestrating (micro)Agents

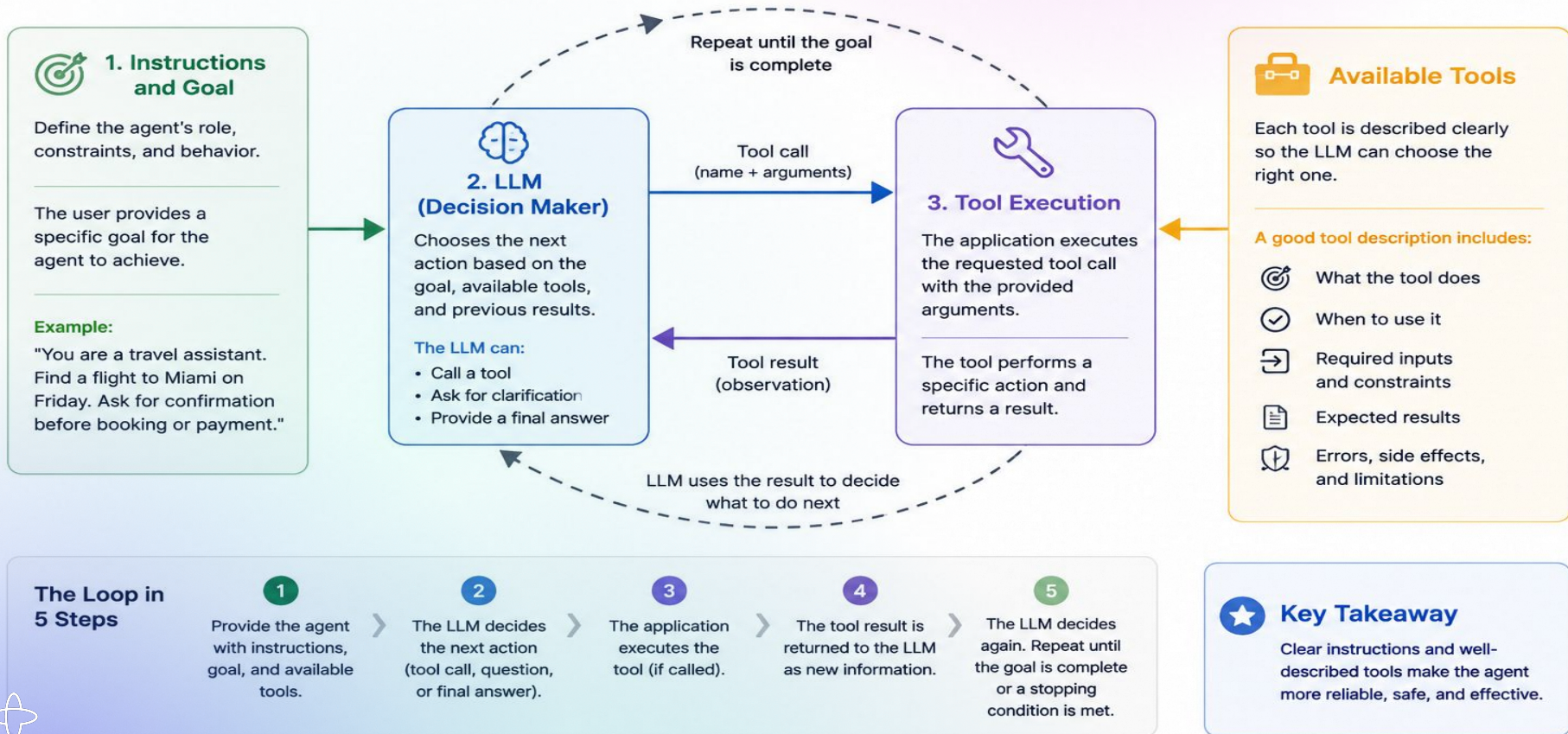


Foundations



The Agentic Loop

An LLM agent observes, decides, acts, and learns—repeating until the goal is complete.



Building a **Durable** Agent





Join at:
[ahaslides.com/
2TOEY](https://ahaslides.com/2TOEY)



Write code as if failure doesn't exist

Distributed systems break, APIs fail, networks flake, and services crash. That's not your problem anymore. Managing reliability shouldn't mean constant firefighting.

Temporal Workflows automatically capture state at every step, and in the event of failure, can pick up exactly where they left off.

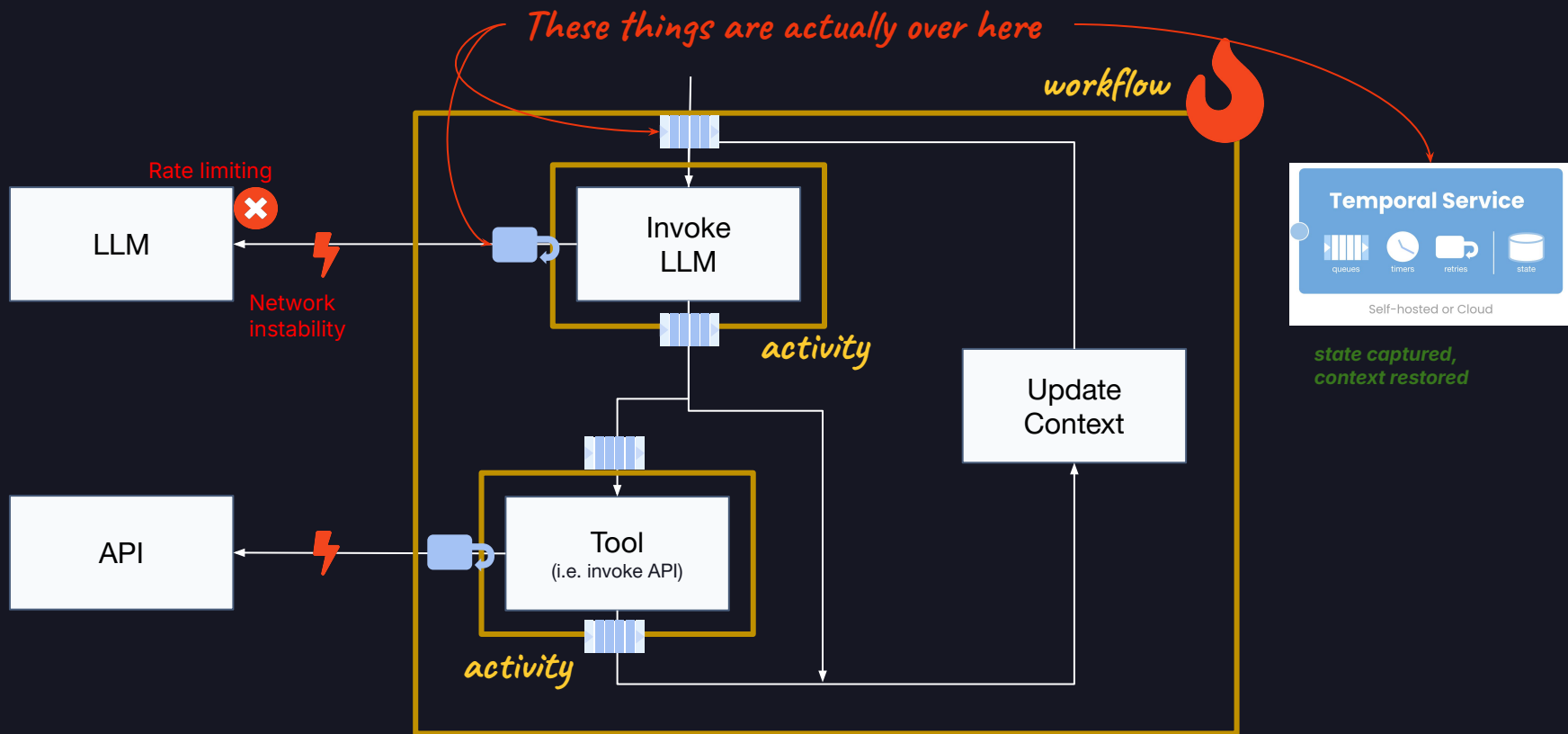
No lost progress, no orphaned processes, and no manual recovery required.

Note: The OpenAI Agents SDK + Temporal solution is currently only available for Python. Typescript coming in the future.

The MIT License

Copyright (c) 2020 Temporal Technologies Inc. All rights reserved.

Copyright (c) 2020 Uber Technologies, Inc.



Agent implementation= **(recoverable)** orchestration process(es) (agentic loop)
+ LLM invocation process(es)
+ tool process(es)
(it's a distributed system)

DEMO #1

OpenAI Agents SDK



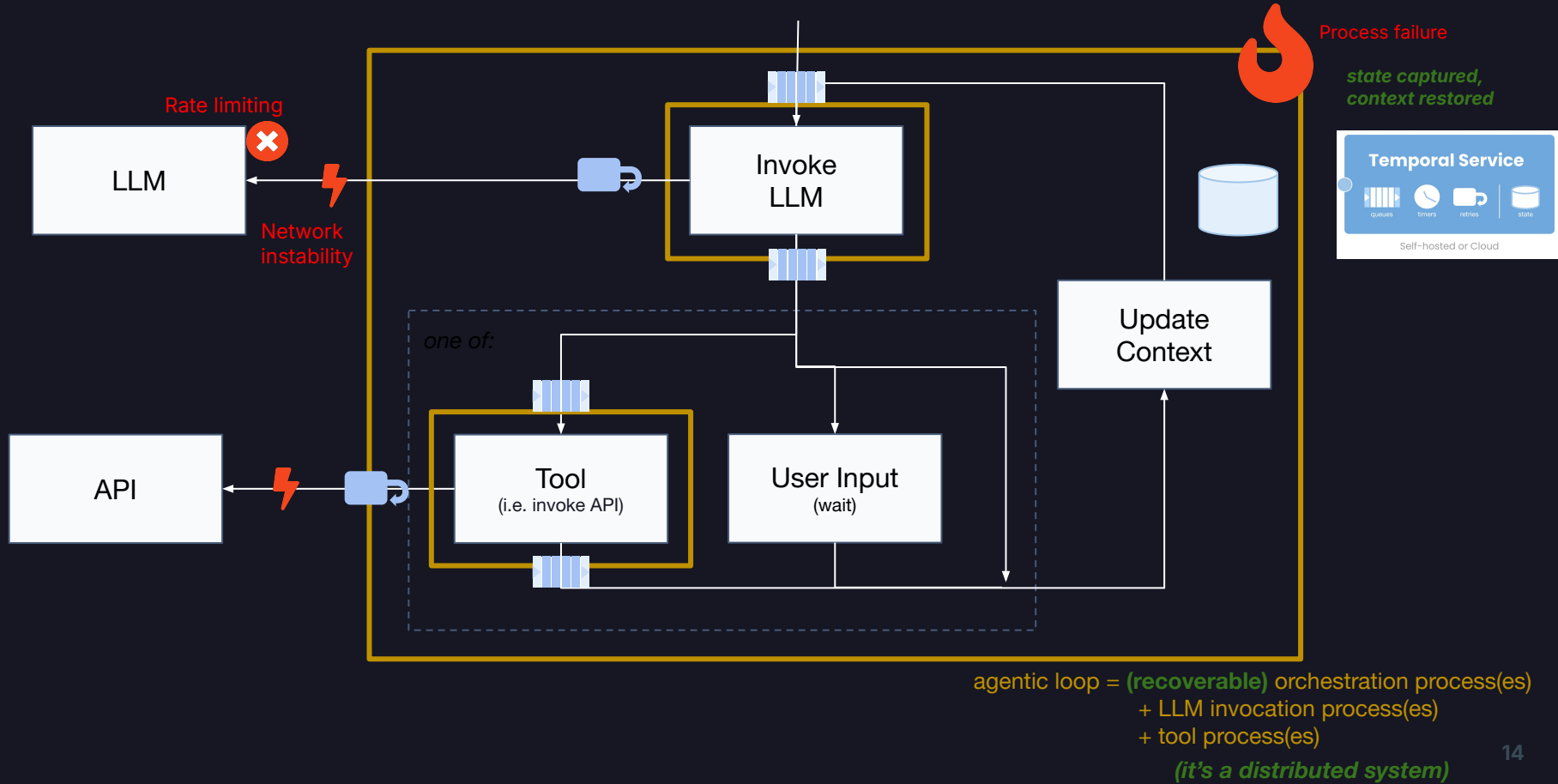
Exercises/Demos

<https://rb.gy/6fsbo3>



HITL





Overall architecture

HUMAN IN THE LOOP

SOME UI

Temporal Client

query READS STATE

```
await handle.query(
    is_input_needed)
await handle.query(
    get_pending_question)
```

signal WRITES STATE

```
await handle.signal(
    provide_user_input,
    response)
```

start_workflow.py:49

TEMPORAL APPLICATION

Query Handler

```
@workflow.query
def get_pending_question(self) -> str:
    return self._question
```

tools_workflow.py:117

Signal Handler

```
@workflow.signal
def provide_user_input(
    self, user_input: str) -> None:
    self._user_input = user_input
    self._input_needed = False
```

tools_workflow.py:110

Application "Main"

WORKFLOW

```
@workflow.run
async def run(self, question: str) -> str:
```

```
@function_tool
async def ask_user(question_text: str):
    self._question = question_text
    self._input_needed = True
```

```
# suspends durably until signalled
await workflow.wait_condition(
    lambda: not self._input_needed)
```

```
answer = self._user_input
self._question = ""
return answer
```

tools_workflow.py:52



DEMO #4

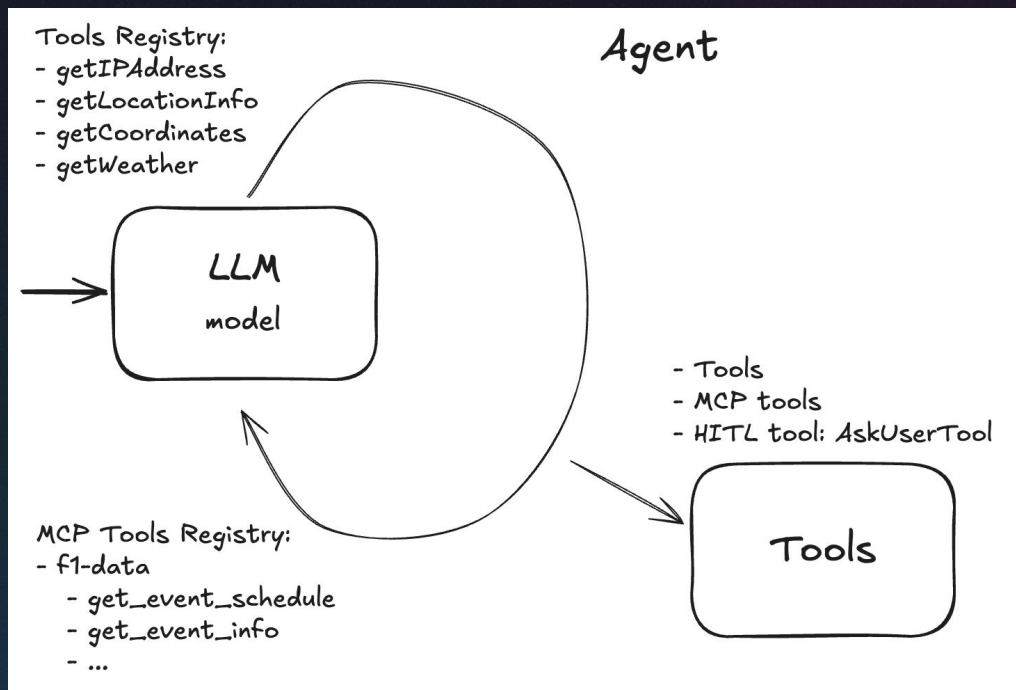
Durable and Resource efficient HITL



Orchestrating (micro)Agents



So far, we've built a single, very busy agent...



Reasons for breaking things up

- Different teams
 - Different release cycles
 - Different tech stacks
- Modular architecture
 - Bulkheads, scale out, ...
- Scoping agents
 - Managing context
- ...

How Contexts Fail...



Context Poisoning

When a hallucination makes it into the context



Context Distraction

When the context overwhelms the training



Context Clash

When parts of the context disagree



Context Confusion

When superfluous context influences the response



...And How to Fix Them



RAG

Add relevant information to assist the LLM



Context Quarantine

Isolate subtask contexts in dedicated threads



Context Summarization

Boil down an accumulated context into a summary



Tool Loadout

Select only relevant tool definitions



Context Pruning

Remove irrelevant or unneeded information



Context Offloading

Store information outside the LLM's context

<https://www.dbreunig.com/2025/07/24/why-the-term-context-engineering-matters.html>

We've seen this before!

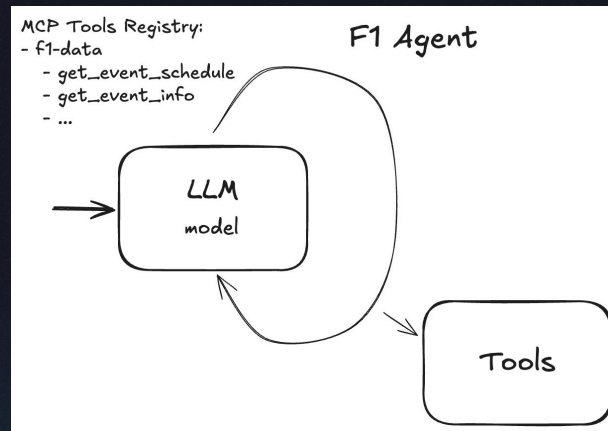
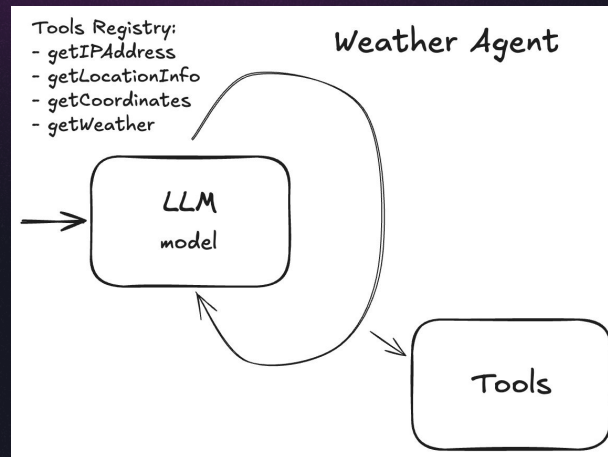
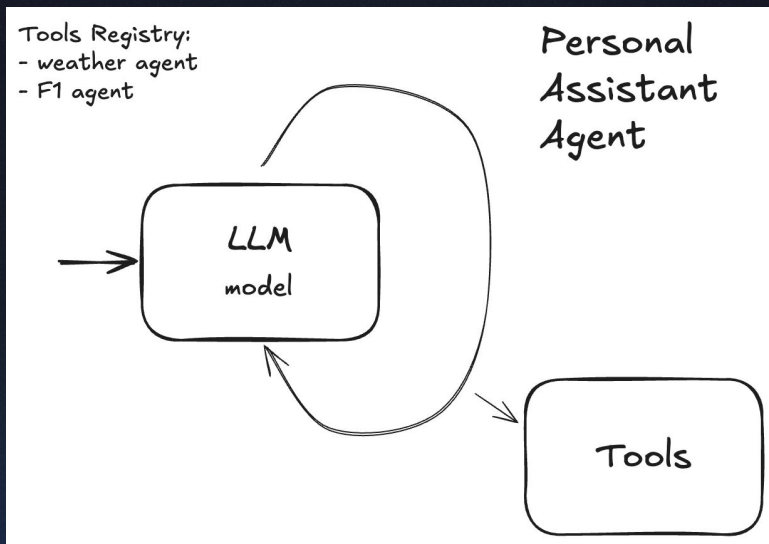
- Different teams
 - Different release cycles
 - Different tech stacks
- Modular architecture
 - Bulkheads, scale out, ...
- Scoping agents
 - Managing context
- ...

Blog



<https://t.mp/micro>

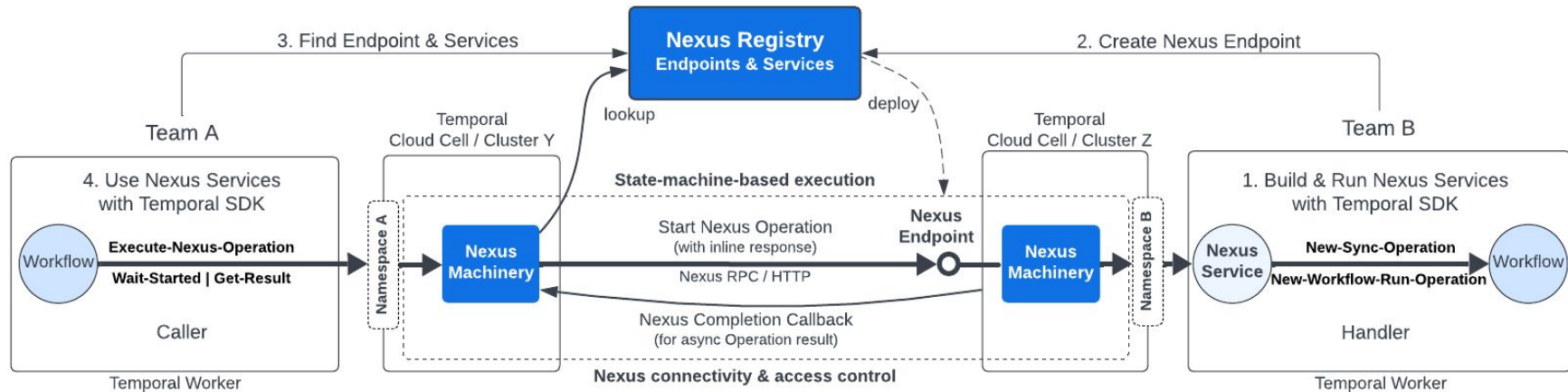
So, let's break this up



There are options for calling sub-agents:

- Call sub-agent via activity
- Sub-agent via child workflow
- Sub-agent via Nexus

Nexus



<https://docs.temporal.io/nexus>

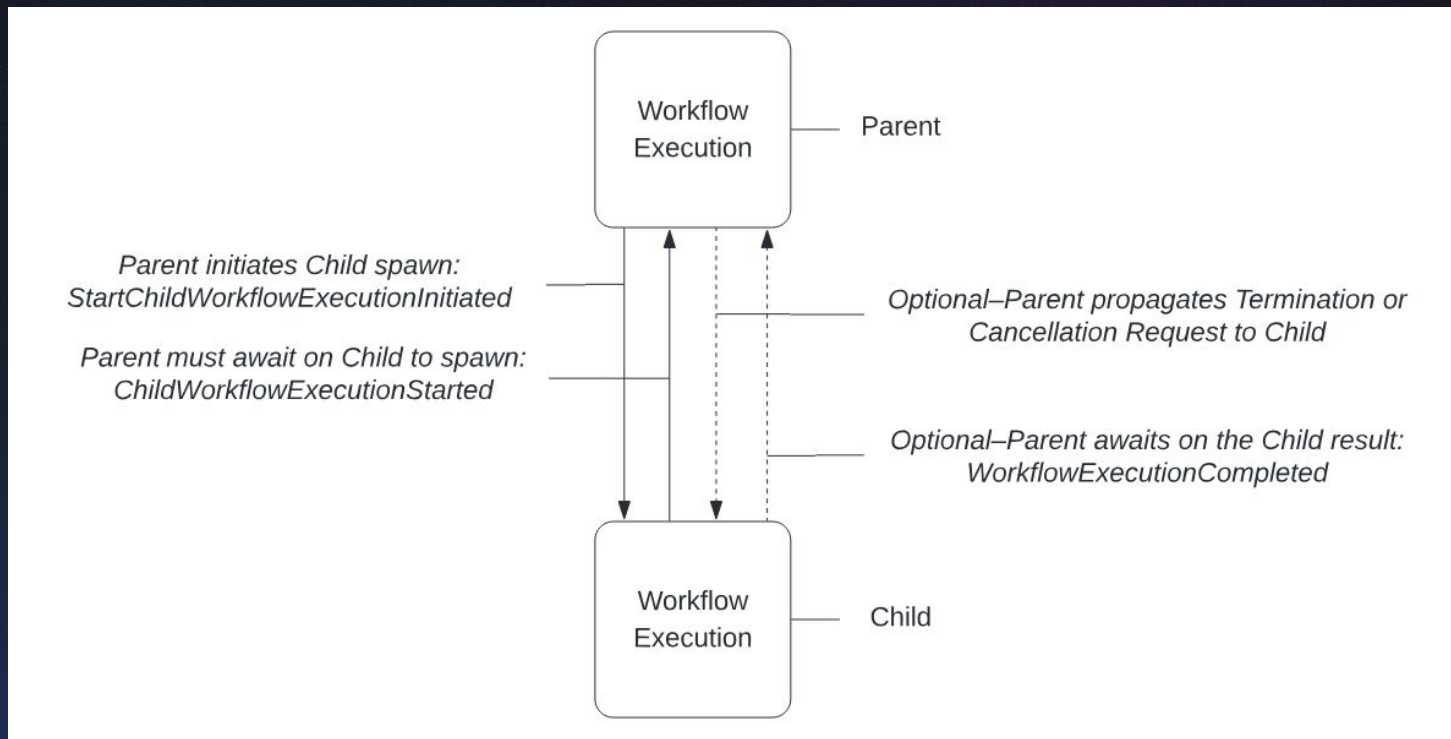


DEMO #5

Multi-agent orchestration: Nexus



Child Workflows:



DEMO #5B

Multi-agent orchestration: Child Workflows



Child workflow

Pros

- Same team, one deploy
- Direct signal + query handle
- Built-in parent-close policy
- Lower latency, no endpoint setup
- Traces propagate today

Cons

- Same namespace, same cluster
- Tight type coupling (both sides import the class)
- Sync result only
- Python-only contract

Nexus

Pros

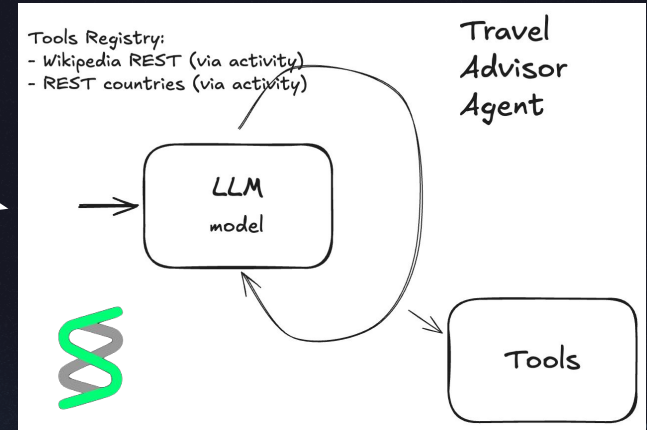
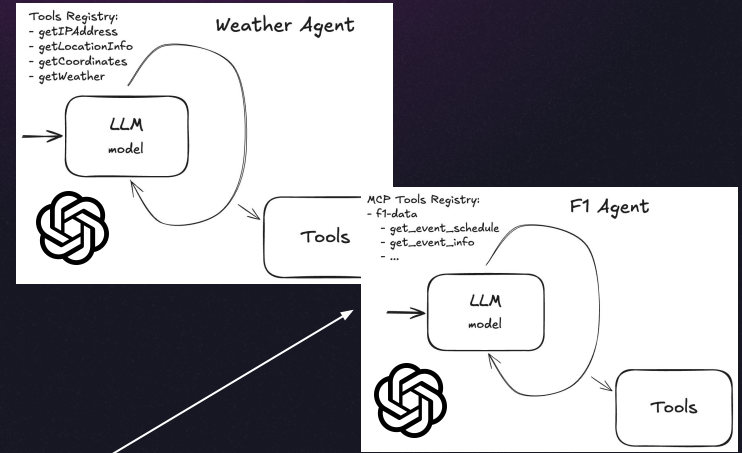
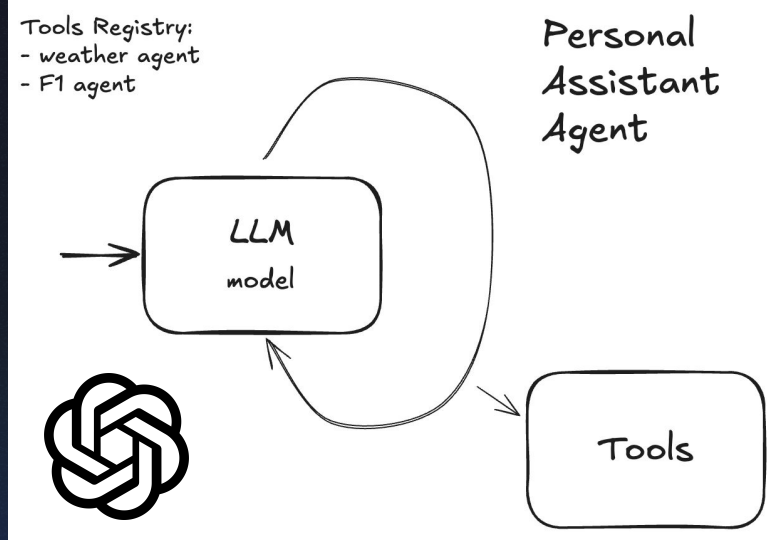
- Crosses namespaces / clusters
- Stable, language-agnostic contract
- Independent deploy cadence
- Endpoint-level policy hooks
- Async completion supported
- Signal/query support via Nexus

Cons

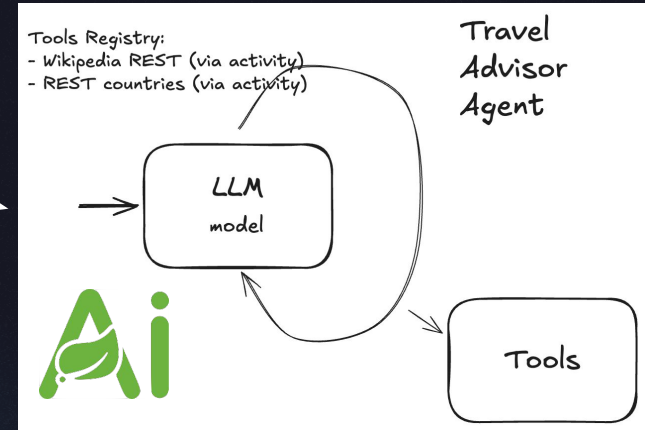
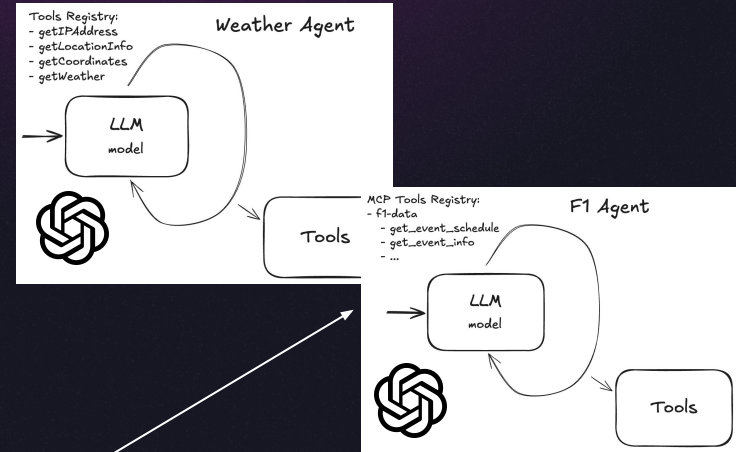
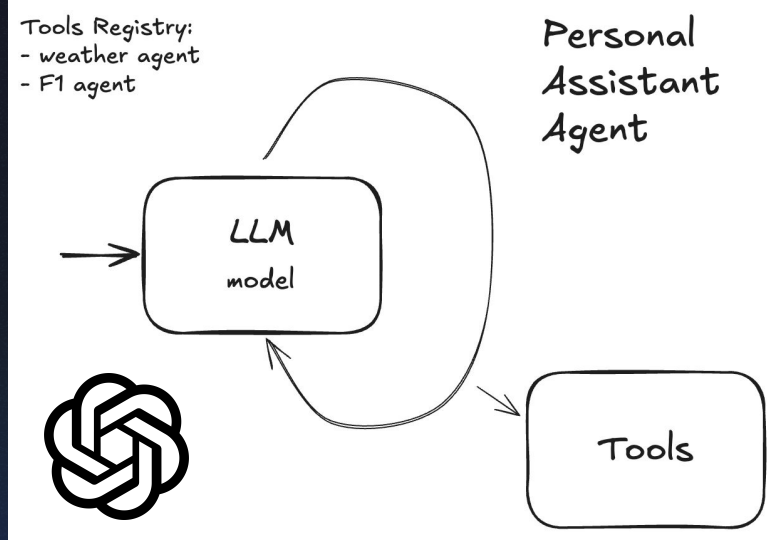
- Endpoint registration per service
- Extra hop latency
- Trace context fragmentation (current Python SDK)



Heterogeneous Agents



Heterogeneous Agents



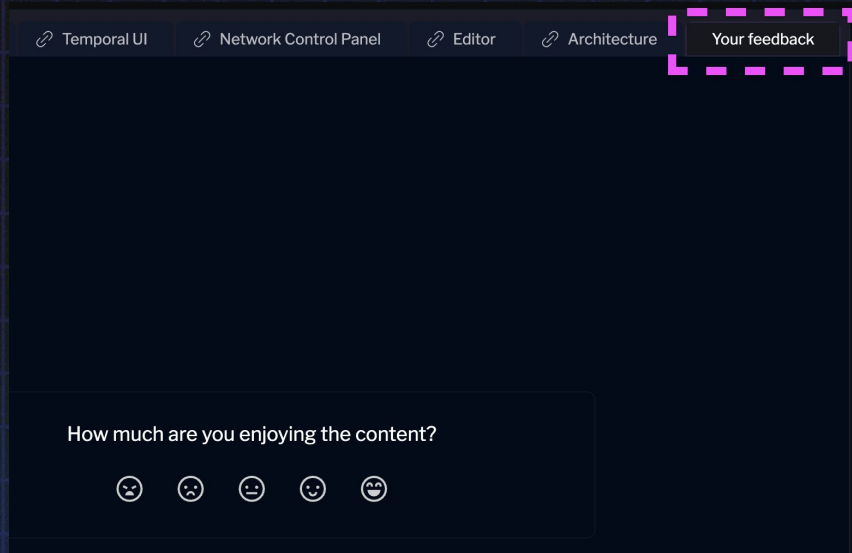
DEMO #6

Multi-agent orchestration: Heterogeneous Agents

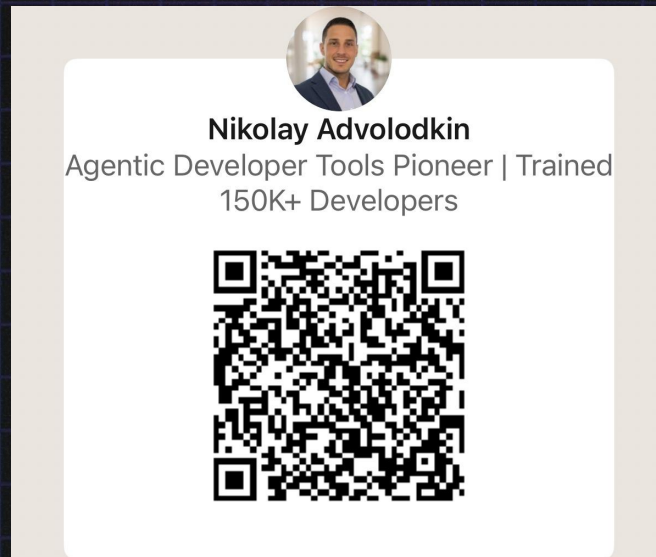
TBD



THANK YOU: FEEDBACK + FOLLOW



The screenshot shows a dark-themed web application with a navigation bar at the top containing links: Temporal UI, Network Control Panel, Editor, and Architecture. A 'Your feedback' button is highlighted with a dashed pink border. Below the navigation bar is a large dark area. At the bottom, a feedback form asks 'How much are you enjoying the content?' and features five emoji icons: 😞, 😟, 😐, 😊, and 😄.



<https://rebrand.ly/link-nikk>



